AIT TALEB Hamza

Rapport Projet : Jeu Flappy Bird sur FPGA

AIT TALEB Hamza - RACHDI Najma

Contents

I. Introduction **1**

II. Travail réalisé **1**

1.Module de Détection d'Impulsions 1
2. Module Bird 3
3.Module X_Obstacle 6
4.Module Y_Obstacle 8
5. Détection des Collisions 9
6.Module VGA 10
7.Module Affichage du score 12
8. Conclusion les rapports de FPGA 14

III.Conclusion **17**

I. Introduction

Dans le cadre d'un projet académique,nous avons entrepris de concevoir un projet qui combine à la fois l'application de concepts théoriques et le développement de compétences pratiques en programmation VHDL. Ce projet consiste à réaliser une version simplifiée et rétro du célèbre **jeu Flappy Bird**. Ce rapport vise à documenter l'ensemble des étapes parcourues pour concevoir et implémenter ce jeu. Nous détaillerons les différentes parties du système, notamment la gestion des mouvements de l'oiseau, la génération et le déplacement des obstacles, la détection de collision, ainsi que l'affichage graphique. Chaque bloc sera expliqué en détail.

II. Travail réalisé

Cette section décrit en détail le processus de développement et d'implémentation. Nous commencerons par présenter les blocs principaux, notamment la gestion du mouvement de l'oiseau, le déplacement et la position des obstacles, ainsi que la synchronisation via un module de gestion de fréquence. Nous explorerons ensuite le bloc VGA, qui est essentiel pour l'affichage graphique du jeu. Enfin, nous détaillerons le top-level design, qui permet d'assurer la communication et l'interaction entre tous les blocs, donnant ainsi vie au jeu dans son ensemble.

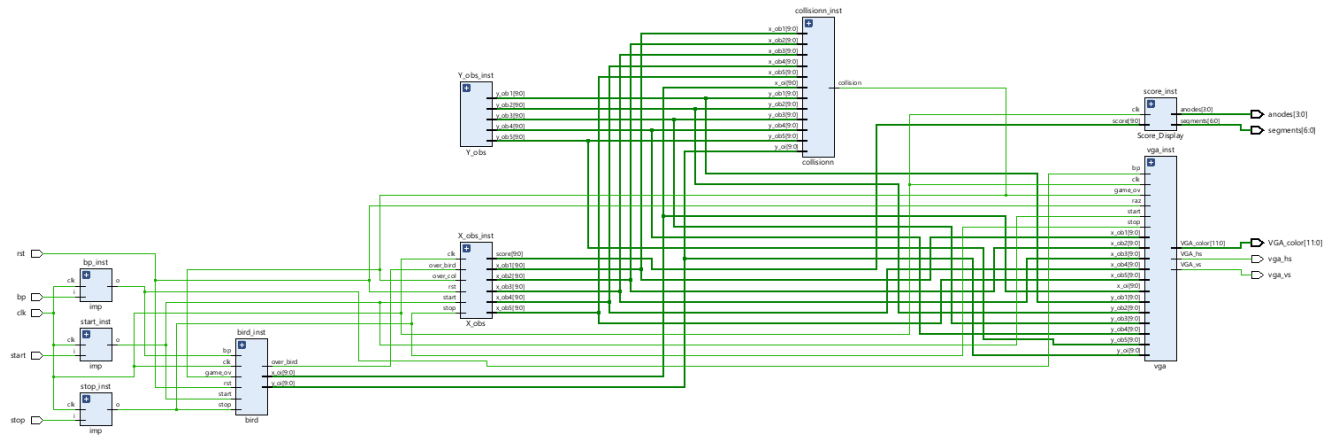


Figure 1: Schéma RTL représentant l'architecture globale du projet Flappy Bird. Ce diagramme illustre les connexions entre les différents modules et leurs interactions.

1.Module de Détection d'Impulsions:

Le module `detection d'impulsions` traite les signaux issus d'un bouton poussoir en appliquant un filtrage anti-rebond et en générant une impulsion propre.

a. Fonctionnalité du Module

Le module remplit deux rôles principaux :

- **Anti-rebond :** Filtrer les oscillations rapides dues au rebond du bouton pour produire un signal stable.
- **Génération d'impulsion :** Créer une impulsion propre et de durée fixe une fois un signal stable détecté.

b. Anti-Rebond : Stabilisation du Signal

La stabilisation du signal est effectuée à l'aide d'un compteur, qui mesure la durée pendant laquelle l'entrée reste constante. Si cette durée dépasse un seuil défini (`DEBOUNCE_TIME`), le signal est considéré comme stable.

```
if i = '1' then
    if debounce_counter < DEBOUNCE_TIME then
        debounce_counter <= debounce_counter + 1;
    else
        stable_signal <= '1'; -- Signal stable d tect
    end if;
else
    debounce_counter <= (others => '0');
    stable_signal <= '0';
end if;
```

Listing 1: Logique anti-rebond

c. Génération d'Impulsion

Dans ce projet, la génération d'impulsion basée sur un appui bouton nécessite de prendre en compte la fréquence de l'horloge système, qui est de **100 MHz**, ce qui correspond à une période de **10 ns par cycle**.

```
if stable_signal = '1' then
    o <= '1';
    pulse_counter <= PULSE_WIDTH;
elsif pulse_counter > 0 then
    pulse_counter <= pulse_counter - 1;
    o <= '1';
else
    o <= '0';
end if;
```

Listing 2: Génération d'impulsion propre

Le diagramme ci-dessous illustre le chemin de données RTL pour la génération d'impulsion, mettant en évidence les interactions entre le signal d'entrée stable et les composants du circuit.

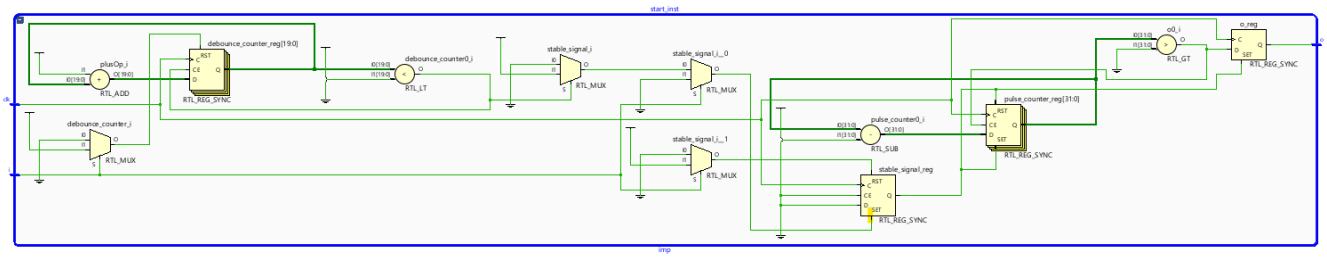


Figure 2: Schéma RTL pour la génération d'impulsion.

d.Consommation des ressources FPGA:

Consommation de ressources estimée pour ce module :

- 53 LUTs sont utilisées principalement pour réaliser les fonctions combinatoires.
- 54 Flip-Flops sont utilisés pour mémoriser des états(des opérations séquentielles).

- Les LUTs sont utilisées pour :

  -La comparaison entre debounce_counter et DEBOUNCE_time

  -L'incrémentation du compteur (debounce_counter + 1).

- Les Flip-Flops sont utilisés pour :

  -Stocker l'état de debounce_counter, stable_signal et o.

  -Maintenir la valeur de pulse_counter.

2. Module Bird

Le module Bird constitue le cœur du jeu Flappy Bird. Il gère le mouvement vertical de l'oiseau, les interactions avec l'utilisateur, et les transitions entre les différentes phases du jeu. Pour cela, il s'appuie sur une machine d'état (FSM) et des calculs basés sur la gravité et les impulsions de saut.

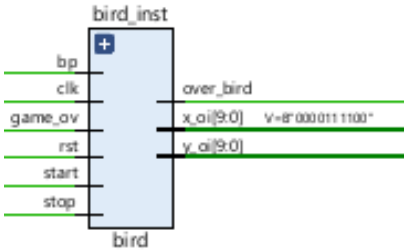
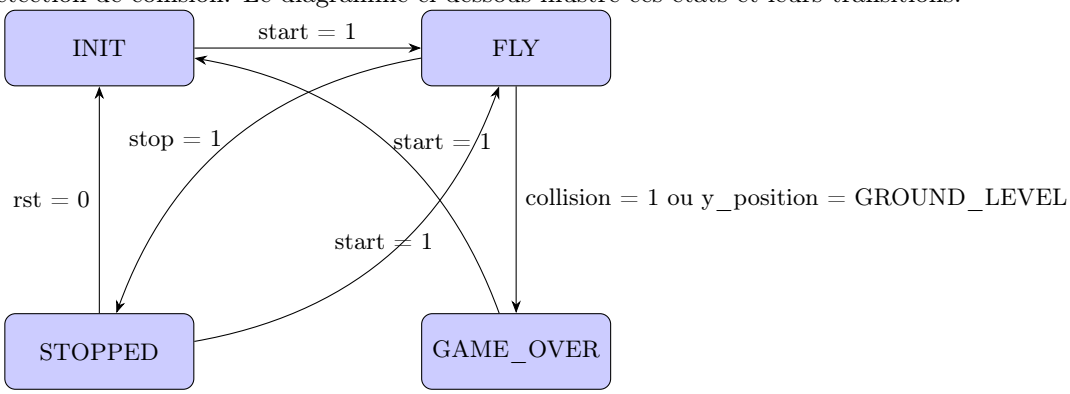


Figure 3: Schéma du module Bird.

a. Diagramme de la Machine d'État

La machine d'état du module Bird est utilisée pour gérer les différents états du jeu en lien avec l'oiseau. Les transitions entre ces états sont déclenchées par des événements tels que l'appui sur un bouton (saut), la gravité, ou la détection de collision. Le diagramme ci-dessous illustre ces états et leurs transitions.



INIT (Initialisation) : Cet état est activé au début du jeu ou après une réinitialisation. L'oiseau est placé à une position verticale définie (x,y)=(60,150), sa vitesse est mise à zéro, et aucun mouvement n'est effectué. Cet état garantit que le jeu commence toujours dans des conditions stables. Une fois que le joueur active le signal start, la FSM passe à l'état FLY.

FLY (En vol) : Cet état simule le mouvement de l'oiseau, influencé par la gravité ou une commande de saut. À chaque cycle d'horloge , la position verticale est mise à jour, permettant un déplacement fluide. Si une collision est détectée ou que l'oiseau touche le sol, l'état passe à GAME_OVER. Si le joueur décide de suspendre le jeu (stop), la FSM passe à STOPPED.

STOPPED (Jeu suspendu) : Dans cet état, l'oiseau reste immobile à sa position actuelle. Cela permet au joueur de mettre le jeu en pause tout en conservant sa progression. L'état peut revenir à FLY pour reprendre le jeu ou à INIT pour recommencer une nouvelle partie.
GAME_OVER (Fin de partie) : Cet état est atteint lorsque l'oiseau touche le sol ou entre en collision avec un obstacle. Il marque la fin du jeu en arrêtant tous les mouvements. Le joueur peut redémarrer une partie en activant le signal start, ce qui ramène la FSM à l'état INIT.

b.Extrait du code

```
if isjumping = '0' then
    if yposition <= GROUNDLEVEL  then
       yposition <= yposition + gravity; // Tombe sous  l effet  de la gravite
    else
        yposition <= GROUNDLEVEL; -- Atteint le sol
        state <= GAMEOVER; // Passe    l'etat GAME_OVER
    end if;
end if;
```

Listing 3: Gestion de la gravité

Dans ce code, la gravité est appliquée uniquement si l'oiseau n'est pas en phase de saut (`is_jumping = '0'`). Lorsque la position de l'oiseau atteint le sol (`GROUND_LEVEL`), le jeu passe à l'état `GAME_OVER`.

Lorsque le joueur déclenche un saut en appuyant sur le bouton (`bp`), une vitesse ascendante est temporairement appliquée à l'oiseau(32 pixels). La vitesse diminue progressivement sous l'effet de la gravité, pour rendre le jeu réaliste. Voici le code correspondant :

```
if is_jumping = '1' then
    if y_position >= SCREEN_HEIGHT + vertical_speed then
        y_position <= y_position - vertical_speed; -- Monte
        vertical_speed <= vertical_speed - "000000001"; -- Diminue la vitesse
    else
        is_jumping <= '0'; -- Fin du saut
    end if;
end if;
```

Listing 4: Gestion du saut

c. Division de l'Horloge :

L'horloge principale du FPGA fonctionne à une fréquence élevée (100 MHz), bien trop rapide pour que les mouvements soient visibles à l'œil nu. Pour résoudre ce problème, un diviseur d'horloge a été implémenté, réduisant la fréquence à 50 Hz. Ce signal d'horloge lente synchronise les mises à jour de la position de l'oiseau, garantissant un mouvement fluide et réaliste.

La valeur du diviseur d'horloge (**DIVISOR**) a été choisie pour obtenir cette fréquence de 50 Hz, en appliquant la formule suivante :

$$\text{DIVISOR} = \frac{f_{\text{clk}}}{2 \times f_{\text{slow_clk}}}$$

où $f_{\text{clk}} = 100\,\text{MHz}$ est la fréquence de l'horloge principale, et $f_{\text{slow_clk}} = 50\,\text{Hz}$ est la fréquence souhaitée. Le calcul donne :

$$\text{DIVISOR} = \frac{100,000,000}{2 \times 50} = 1,000,000$$

Cette valeur garantit une réduction cohérente de la fréquence d'horloge pour que les mouvements du jeu soient perceptibles à l'œil humain .

Voici le code VHDL correspondant à cette division d'horloge :

```
process(clk, rst)
begin
    if rst = '0' then -- rst active
        clk_div_count <= (others => '0');
        slow_clk <= '0';
    elsif rising_edge(clk) then
        if clk_div_count = DIVISOR then
            clk_div_count <= (others => '0');
            slow_clk <= not slow_clk;
        else
            clk_div_count <= clk_div_count + 1;
        end if;
    end if;
end process;
```

Listing 5: Division de l'horloge pour obtenir 50 Hz

d.Schéma RTL du Module Bird

Le schéma RTL du module Bird met en évidence –chi haja .

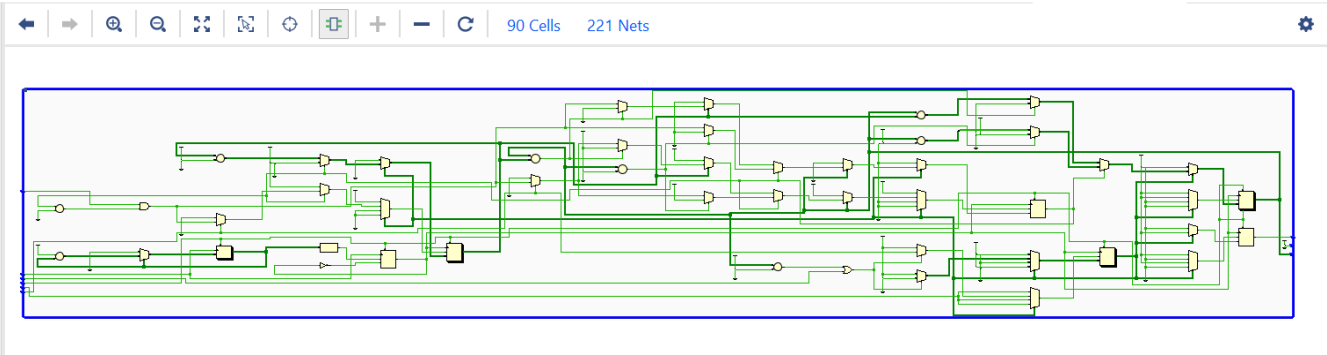


Figure 4: Schéma RTL du Module Bird. Les blocs principaux incluent la gestion de la gravité, la logique de saut, et les registres de position.

e.Consommation des ressources FPGA:

Consommation des ressources FPGA La consommation des ressources pour le module bird est estimée comme suit :

- 112 Slice LUTs .
- 45 Flip-Flops .

- Utilisation des LUTs:

  - La logique combinatoire dans la machine à états finis (FSM), en particulier pour gérer les transitions entre les états INIT, FLY, STOPPED, et GAME_OVER.

  - Les comparaisons (par exemple : y_position >= SCREEN_HEIGHT, stop = '1').

  - Les calculs arithmétiques pour mettre à jour la position verticale de l'oiseau (y_position, vertical_speed) ou pour générer le signal slow_clk via un diviseur d'horloge.

- Utilisation des Flip-Flops:

  - Stocker l'état courant de la FSM (state).

- Mémoriser la position verticale de l'oiseau (y_position).
- Conserver la vitesse verticale (vertical_speed) et l'état de saut (is_jumping).

3.Module X_Obstacles :

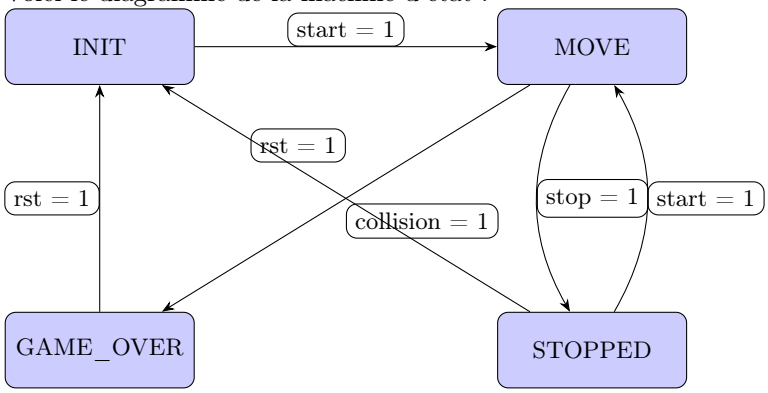
Le module `X_Obstacle` gère le déplacement horizontal des obstacles dans le jeu Flappy Bird. Ces obstacles, qui défilent de droite à gauche à une vitesse constante, représentent le principal défi pour le joueur. Ce module utilise une machine d'état (FSM) pour organiser les différentes phases du jeu et un processus de mise à jour des positions pour assurer un mouvement fluide.

a. Machine d'État (FSM)

La machine d'état (FSM) est composée de quatre états principaux :

- **INIT (Initialisation)** : Définit les positions initiales des obstacles et attend le signal `start`.
- **MOVE (Déplacement)** : Met à jour les positions des obstacles à chaque cycle d'horloge lente.
- **STOPPED (Jeu suspendu)** : Suspend le mouvement lorsque le signal `stop` est activé.
- **GAME_OVER (Fin de partie)** : Arrête le déplacement en cas de collision ou de fin de jeu.

Voici le diagramme de la machine d'état :



b. Déplacement horizontal des Obstacles:

Le déplacement des obstacles est simulé en mettant à jour leurs positions à chaque cycle d'horloge lente. Lorsqu'un obstacle atteint le bord gauche de l'écran, il est repositionné derrière le dernier obstacle pour maintenir la continuité.

```
if x_position1 = 0 then
    x_position1 <= x_position5 + OBSTACLE_SPACING;
else
    x_position1 <= x_position1 - 1;
end if;
```

Listing 6: Déplacement des obstacles

c. Division de l'Horloge

Comme pour le module Bird, une horloge lente est utilisée pour synchroniser le déplacement des obstacles à une vitesse visible pour le joueur. Cela permet de garantir que les mouvements ne sont pas trop rapides et restent perceptibles. La division d'horloge est configurée pour produire une fréquence adaptée au défilement fluide des obstacles.

d.Consommation des ressources FPGA

La consommation des ressources pour le module X_obs est estimée comme suit :
-345 Slice LUTs.
- 85 Flip-Flops.

- Utilisation des LUTs:

  -La logique combinatoire dans la machine à états finis (FSM), gérant les transitions entre les états INIT, FLY, STOPPED, et GAME_OVER.

  -Les comparaisons nécessaires pour déplacer les positions des obstacles horizontalement.

  -Les calculs pour ajuster la position des obstacles (décrémentation des positions avec l'espacement constant).
  -La gestion de l'incrémentation du score à chaque cycle.

- Utilisation des Flip-Flops:

  -Stocker l'état courant de la FSM (state).

  -Mémoriser les positions des cinq obstacles (x_position1 à x_position5).

  -Conserver la valeur courante du score (s_score).

e. Simulation avec un testbench

Pour valider le fonctionnement de notre bloc `X_obstacle` et tester simultanément l'évolution du score, nous avons réalisé un testbench. Cela nous a permis d'observer le comportement du module dans des conditions simulées avec une horloge lente(slow_clk).
La simulation a montré un fonctionnement conforme aux attentes :

- Lorsque le signal `start` est à 0, les obstacles conservent leurs positions initiales (prédéfinies), et le score reste à 0.

- Dès qu'un signal `start` est détecté, les obstacles commencent à défiler vers la gauche, ce qui se traduit par une diminution progressive de leurs positions `X_obstacle` à chaque front montant du signal `slow_clk`.

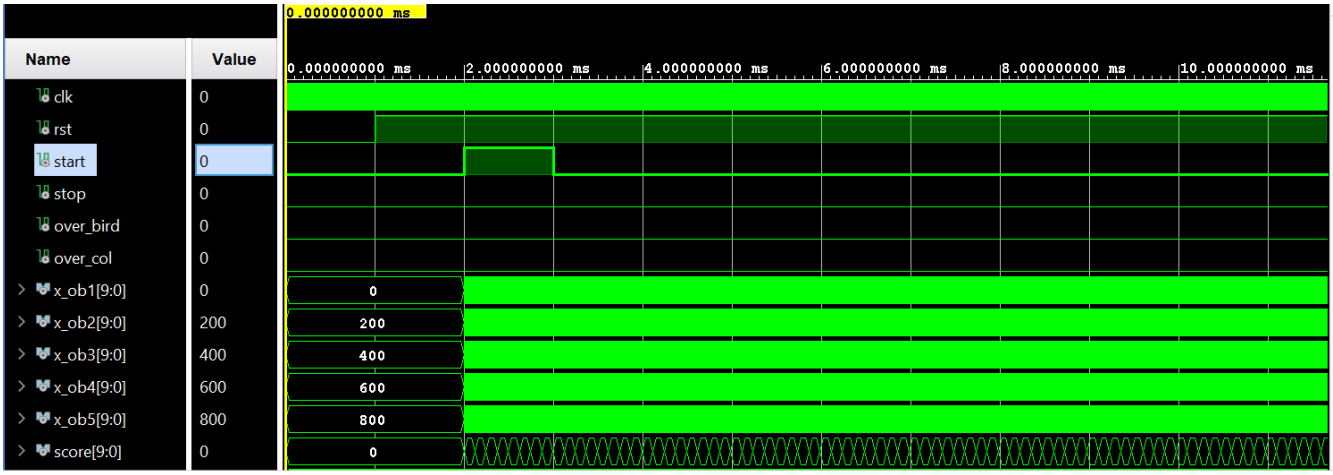


Figure 5: Simulation au démarrage : Obstacles en mouvement et score initialisé.

De plus, nous avons observé que le score augmente de manière incrémentale chaque fois qu'un obstacle atteint la position 0. Par ailleurs, le score reste constant lorsque le signal `stop` est activé ou lorsqu'une collision est détectée, conformément à la logique du jeu.

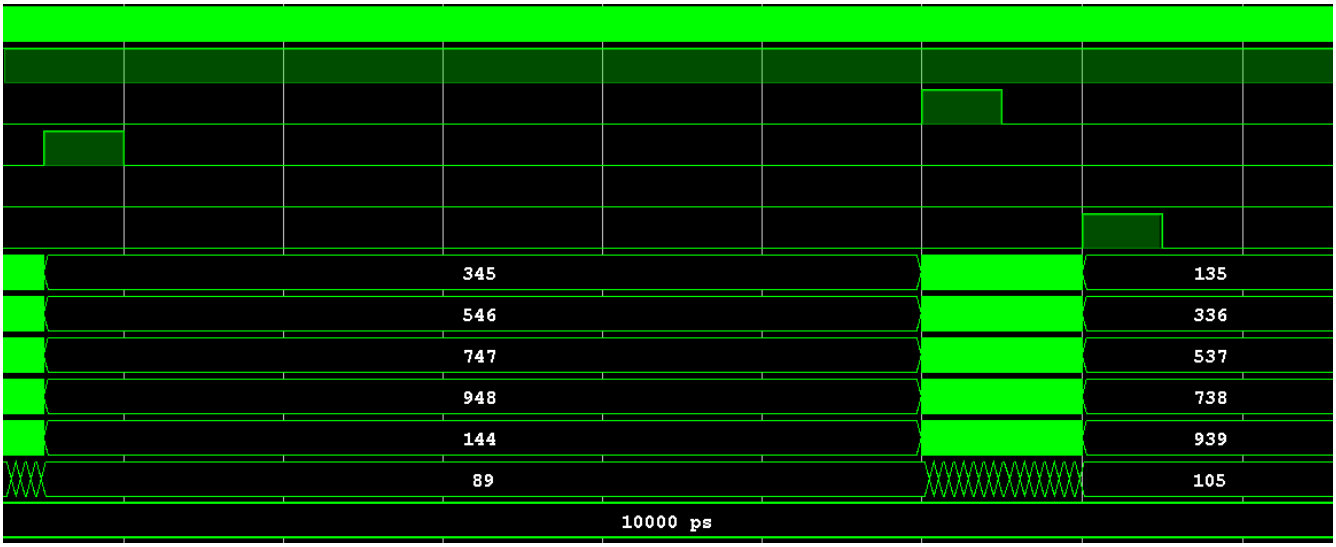


Figure 6: Simulation lors d'un signal `stop` ou d'une collision.

4.Module Y_Obstacle :

Le module `Y_Obstacle` est responsable de la gestion des positions verticales fixes des ouvertures dans les obstacles du jeu Flappy Bird. Contrairement au module `X_Obstacle`, qui traite le mouvement horizontal, ce module se concentre exclusivement sur l'assignation de hauteurs prédéfinies pour les cinq obstacles, garantissant une disposition structurée et cohérente.

a. Fonctionnement du Module

Les obstacles dans Flappy Bird possèdent des ouvertures statiques permettant à l'oiseau de passer sans collision. Les positions verticales de ces ouvertures sont définies par des constantes prédéterminées pour chaque obstacle. Ces constantes garantissent que les hauteurs restent fixes tout au long du jeu, assurant un équilibre entre prévisibilité et défi.

Voici un extrait du code déclarant ces constantes :

```
constant Y_POS1 : unsigned(9 downto 0) := to_unsigned(150, 10);
constant Y_POS2 : unsigned(9 downto 0) := to_unsigned(120, 10);
constant Y_POS3 : unsigned(9 downto 0) := to_unsigned(180, 10);
constant Y_POS4 : unsigned(9 downto 0) := to_unsigned(90, 10);
constant Y_POS5 : unsigned(9 downto 0) := to_unsigned(130, 10);
```

Listing 7: Constantes pour les positions verticales

Ces valeurs correspondent aux hauteurs des ouvertures pour chaque obstacle, exprimées en pixels.

b. Assignation des Positions

Les constantes définies précédemment sont assignées aux signaux de sortie `y_ob1` à `y_ob5`, représentant les hauteurs des obstacles. Voici un exemple de cette assignation :

```
y_ob1 <= std_logic_vector(Y_POS1);
y_ob2 <= std_logic_vector(Y_POS2);
y_ob3 <= std_logic_vector(Y_POS3);
y_ob4 <= std_logic_vector(Y_POS4);
y_ob5 <= std_logic_vector(Y_POS5);
```

Listing 8: Assignation des positions fixes

Cette assignation garantit que les obstacles apparaissent toujours aux mêmes hauteurs, ce qui simplifie la logique du module et maintient une difficulté constante.

5. Détection des Collisions:

Le module `collision_detector` gère la détection des collisions entre l'oiseau et les obstacles, ainsi que le calcul du score. Il joue un rôle clé dans la logique du jeu Flappy Bird.

a. Fonctionnalité Principale

Ce module vérifie si l'oiseau entre en contact avec une partie solide d'un obstacle.

La logique de détection de collision compare les coordonnées X et Y de l'oiseau avec celles des obstacles. Une collision est détectée si les positions se chevauchent et si l'oiseau n'est pas entièrement dans l'ouverture de l'obstacle.

```
if (
    bird_x + BIRD_WIDTH > obs_x1 and bird_x < obs_x1 + OBSTACLE_WIDTH and
    not (bird_y >= obs_y1 and bird_y + BIRD_HEIGHT <= obs_y1 + HOLE_HEIGHT)
) then
    collision <= '1'; -- Collision detected
end if;
```

Listing 9: Logique de détection des collisions

b. Consommations des ressources FPGA

Le module *collision* ne figure pas explicitement dans le tableau de synthèse. Cela s'explique par le fait qu'une optimisation est gérée automatiquement par le logiciel Vivado. La figure ci-dessus montre que le module *collision* ne contient que des portes logiques. Par conséquent, nous supposons qu'il consomme uniquement des **Slice LUTs (Look-Up Tables)**, utilisées pour réaliser les comparaisons et les portes logiques nécessaires à la détection des collisions.

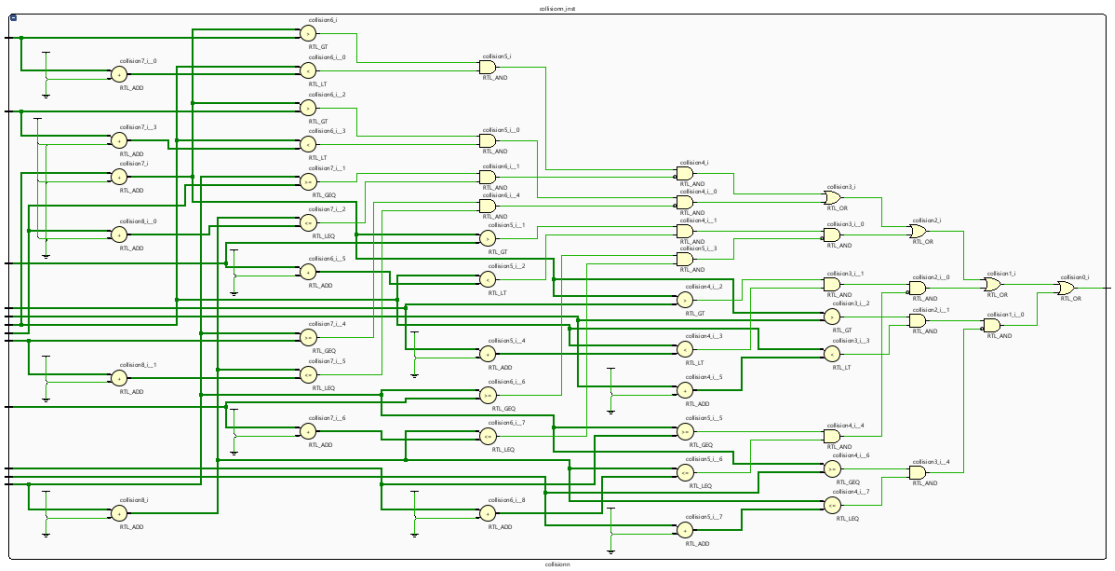


Figure 7: Schéma RTL du module *collision*

6.Module VGA:

Le module `VGA` gère l'affichage graphique du jeu Flappy Bird sur un écran VGA standard. Il est constitué de plusieurs sous-modules spécialisés qui assurent la coordination des différents éléments du jeu.

a. Fonctionnalité Globale

Le module `VGA` se charge de :

- Générer les signaux de synchronisation VGA (`VGA_hs`, `VGA_vs`).
- Afficher les éléments graphiques, tels que l'oiseau, les obstacles, et les textes "Start" et "Game Over".
- Gérer la mémoire vidéo pour un affichage fluide.

b. Sous-Modules

b.1 Address Counter : Ce sous-module génère les coordonnées X et Y pour chaque pixel à afficher. Il utilise deux compteurs : un pour la direction horizontale (`count_x`) et un pour la direction verticale (`count_y`).

```
process(clk)
begin
    if rising_edge(clk) then
        if (raz = '0') then
            count_x <= 0;
            count_y <= 0;
        elsif (count_x = 639) then
            count_x <= 0;
            if (count_y = 479) then
                count_y <= 0;
            else
                count_y <= count_y + 1;
            end if;
        else
```

```
            count_x <= count_x + 1;
        end if;
    end if;
end process;
```

Listing 10: Comptage des pixels dans Address Counter

Explication :

- `count_x` s'incrémente jusqu'à 639 (largeur de l'écran 640), puis se réinitialise.
- Lorsque `count_x` se réinitialise, `count_y` s'incrémente pour passer à la ligne suivante.
- Une fois `count_y` à 479 (hauteur de l'écran 480), l'écran est complètement balayé.

b.2. VGA Bitmap :

Le module `VGA_Bitmap` est utilisé pour gérer l'affichage graphique sur un écran VGA avec une résolution standard de **640x480 pixels**. Il permet de gérer efficacement les données graphiques en mémoire et de produire les signaux nécessaires pour l'affichage.

b.3 ROM Start et ROM GAME_OVER:

Le module `Rom_start` affiche le texte "Start" en début de partie, en utilisant une mémoire ROM pour stocker les données graphiques nécessaires à l'affichage. De même, le module `Rom_GAME_OVER` affiche le texte "GAME_OVER" à la fin d'une partie, signalant ainsi la fin du jeu.

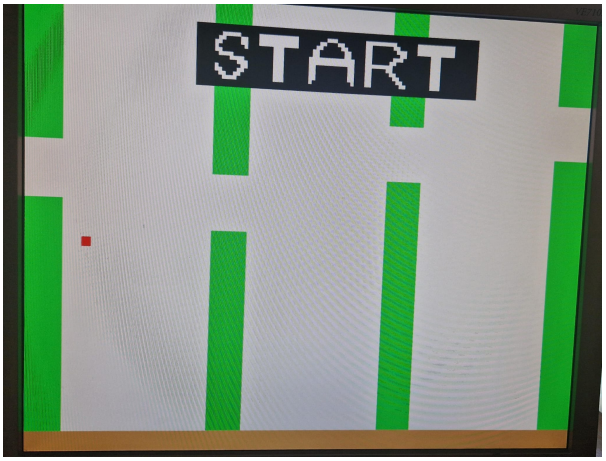


Figure 8: Affichage du texte "GAME_OVER".

Figure 9: Affichage du texte "Start".

c. Consommation des ressources FPGA pour le module VGA

Le tableau ci-dessous présente la consommation des ressources FPGA pour le module *vga* et ses sous-modules. Ce module regroupe plusieurs sous-modules spécifiques qui gèrent des aspects essentiels de l'affichage graphique. Voici une analyse détaillée des ressources consommées :

Name 1	Slice LUTs (63400)	Slice Registers (126800)	F7 Muxes (31700)	Block RAM Tile (135)	Bonded IOB (210)	BUFGCTRL (32)
vga_inst (vga)	406	328	2	60	0	0
Compteur (addre:	201	65	0	0	0	0
game_over_inst (g	41	37	1	0	0	0
start_inst (start_ro	30	35	1	0	0	0
vga (VGA_bitmap	126	183	0	60	0	0

Figure 10: Consommation des ressources FPGA pour le module *vga*

Analyse des ressources utilisées :

- **Slice LUTs (406 LUTs) :**
  - Réalisation des fonctions combinatoires pour la synchronisation VGA, les calculs des coordonnées des pixels, et la génération des couleurs pour les différents éléments graphiques (oiseau, obstacles, arrière-plan,..).
  - Gestion de la logique combinatoire des sous-modules : address_counter, VGA_bitmap, game_over_inst, et start_inst.
- **328 Flip-Flops :**
  - Stockage des états séquentiels pour la synchronisation horizontale et verticale.
  - Gestion des données de pixels pour les transitions entre les états d'affichage (écrans "Start" et "Game Over").
  - Maintien des coordonnées des pixels pour chaque cycle d'horloge.
- **Block RAM Tiles(60 BRAM) :**
  - Stockage des données graphiques pour les écrans "Game Over" et "Start".
  - Mémoire utilisée pour conserver les données des pixels du jeu.
- **F7 Muxes (2 Muxes) :**
  - Combinaison logique des signaux provenant des différents sous-modules pour garantir un affichage VGA cohérent.

7.Module Affichage du score:

Le module `Score_Display` permet d'afficher un score binaire de 10 bits sur 4 afficheurs 7 segments. Voici les principales étapes de son fonctionnement :

a. Conversion Binaire vers BCD Le score binaire est converti en **BCD** (*Binary-Coded Decimal*), ce qui permet de représenter chaque chiffre du score sur 4 bits (de 0 à 9). Cela facilite son affichage sur les 4 digits.Nous avons décidé limiter le score à 999.

```
if score > "1111100111" then -- Si le score est > 999
    bcdscore <= "0000001001100111"; -- Limite    999
else
    bcd_score(15 downto 12) <= std_logic_vector(to_unsigned(to_integer(unsigned(score)) / 1000, 4));
    bcd_score(11 downto 8)  <= std_logic_vector(to_unsigned((to_integer(unsigned(score)) mod 1000) / 100, 4));
    bcd_score(7 downto 4)   <= std_logic_vector(to_unsigned((to_integer(unsigned(score)) mod 100) / 10, 4));
    bcd_score(3 downto 0)   <= std_logic_vector(to_unsigned(to_integer(unsigned(score)) mod 10, 4));
end if;
```

Listing 11: Conversion binaire en BCD

Exemple de calcul : Pour un score binaire de `1011011011` (soit 731 en décimal) :

- **Milliers** : $\lfloor 731 \div 1000 \rfloor = 0$,
- **Centaines** : $\lfloor (731 \mod 1000) \div 100 \rfloor = 7$,
- **Dizaines** : $\lfloor (731 \mod 100) \div 10 \rfloor = 3$,
- **Unités** : $731 \mod 10 = 1$.

Résultat en BCD : `0000 0111 0011 0001`.

b. Multiplexage des afficheurs Pour afficher 4 chiffres avec un nombre limité de broches, un multiplexeur est utilisé. Chaque afficheur est activé successivement.

```
process(clk)
begin
    if rising_edge(clk) then
        refresh_counter <= refresh_counter + 1;
        if refresh_counter = 65535 then
            refresh_counter <= (others => '0');
            if current_digit = 3 then
                current_digit <= "00";
            else
                current_digit <= current_digit + 1;
            end if;
        end if;
    end if;
end process;
```

Listing 12: Multiplexage des afficheurs

La fréquence de multiplexage est calculée comme suit :

$$f_{\text{multiplexage}} = \frac{f_{\text{horloge}}}{2^{16}} = \frac{100\,\text{MHz}}{65536} \approx 1.52\,\text{kHz}.$$

Avec 16 le nombre de bits minimal pour afficher 999.

c. Décodage des segments Chaque chiffre BCD est converti en un motif pour les segments (a-g) de l'afficheur. Par exemple, pour afficher le chiffre `2`, le motif activé est `0010010`.

```
case digit_value is
    when "0000" => segments <= "0000001"; -- 0
    when "0001" => segments <= "1001111"; -- 1
    when "0010" => segments <= "0010010"; -- 2
    when "0011" => segments <= "0000110"; -- 3
    ...
end case;
```

Listing 13: Décodage des segments

d. Activation des anodes Chaque chiffre est affiché sur son anode correspondante selon le multiplexage :

```
case current_digit is
    when "00" => anodes <= "1110"; -- Activer afficheur 1
    when "01" => anodes <= "1101"; -- Activer afficheur 2
    when "10" => anodes <= "1011"; -- Activer afficheur 3
    when "11" => anodes <= "0111"; -- Activer afficheur 4
end case;
```

Listing 14: Activation des anodes

e.Gestion des ressources FPGA

Le module utilise :
-9 LUT
-18 filp flops

- Slice LUTs (Look-Up Tables) :
  - Conversion binaire-vers-BCD.
  - Décodage des segments.
  - Logique du multiplexage.
- Slice Registers (Flip-Flops):
  - Stockage du compteur de rafraîchissement.
  - Stockage du chiffre actif (`current_digit`).
  - Enregistrement du score converti (`bcd_score`).

Conclusion

Le module `Score_Display` combine une conversion binaire-vers-BCD, un multiplexage efficace et un décodage précis pour permettre un affichage fluide et lisible d'un score jusqu'à 999. Sa conception optimise l'utilisation des ressources FPGA tout en offrant une fonctionnalité essentielle pour les applications embarquées interactives.

8. Conclusion les rapports de FPGA

a. Conclusion sur la consommation des ressources FPGA

Resource	Utilization	Available	Utilization %
LUT	1051	63400	1.66
LUTRAM	3	19000	0.02
FF	638	126800	0.50
BRAM	60	135	44.44
IO	30	210	14.29

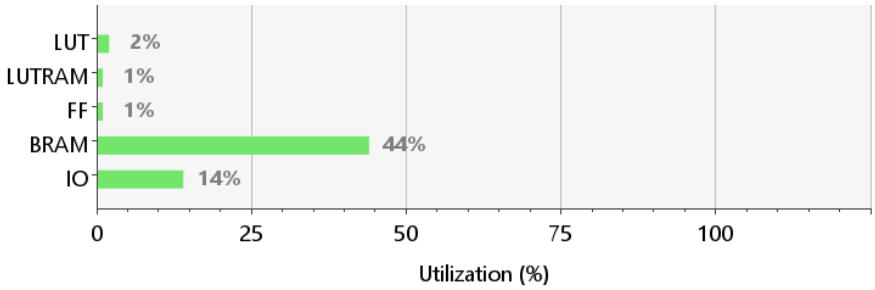


Figure 11: Utilisation des ressources FPGA

Le tableau ci-dessus montre que :

- **LUT (Look-Up Tables)** : 869 LUTs sont utilisées, ce qui représente **1.37%** de la capacité totale. Ces LUTs sont nécessaires pour les calculs combinatoires, comme la gestion des comparaisons, des conditions et des logiques utilisées dans les modules.
- **LUTRAM** : La consommation est très faible, avec seulement **3 LUTRAMs** utilisés, soit **0.02%** de la capacité totale. Ces ressources sont peu sollicitées dans notre projet, car elles sont utilisées pour des fonctions de stockage temporaire limitées.
- **FF (Flip-Flops)** : **638 Flip-Flops** sont utilisés, ce qui représente seulement **0.5%** de la capacité totale. Ces Flip-Flops permettent de stocker les états séquentiels, comme les coordonnées des pixels ou les états intermédiaires des signaux.
- **BRAM (Block RAM)** : La consommation des BRAM est relativement élevée, atteignant **44.44%**, avec **60 blocs utilisés sur 135 disponibles**. Cette utilisation est due au stockage des données graphiques, comme les écrans "Start" et "Game Over", ainsi que les informations nécessaires pour l'affichage des éléments du jeu (oiseau, obstacles, arrière-plan).
- **IO (Input/Output)** : Les broches d'entrée/sortie (IO) sont utilisées à hauteur de **14.29%**, soit **30 IOs** sur 210 disponibles. Elles servent principalement à connecter les périphériques externes, comme les boutons d'entrée ou la sortie VGA.

Conclusion : La consommation globale des ressources FPGA est très faible, sauf pour les blocs BRAM, qui sont fortement sollicités à cause des besoins de stockage graphique. Cette faible utilisation montre qu'il reste beaucoup de ressources disponibles pour ajouter de nouvelles fonctionnalités ou améliorer le design.

b. Rapport Timing

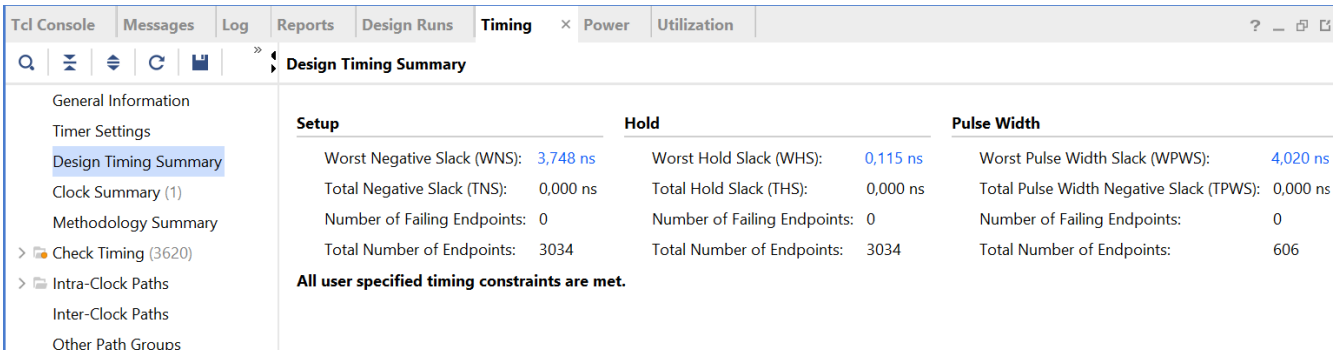


Figure 12: Rapport Timing du projet.

Le rapport timing montre que toutes les contraintes temporelles imposées au design sont respectées. Cela signifie que les signaux critiques du projet, tels que ceux liés au contrôle du VGA et à la synchronisation des obstacles, sont générés sans dépasser les délais admissibles. En dépit des limitations de temps pour développer ce projet, cette validation timing garantit la fiabilité et la stabilité du système lors de son exécution sur FPGA.

III.Conclusion

Ce projet Flappy Bird a représenté une expérience enrichissante et formatrice, malgré les défis rencontrés, notamment le temps limité à consacrer à son développement. Nous avions initialement l'objectif ambitieux de créer une machine d'état globale pour gérer l'intégralité du jeu et d'afficher le score directement sur l'écran VGA. Cependant, ces fonctionnalités n'ont pas pu être finalisées dans les délais impartis. À la place, nous avons choisi de nous concentrer sur les éléments essentiels, tels que l'affichage des obstacles avec des trous fixes et la gestion du score via un afficheur 7 segments.
L'exploration de l'affichage graphique a été particulièrement captivante, étant donné que c'était notre première expérience avec un module VGA. Concevoir les visuels du jeu, comme les textes "Start" et "Game Over", ainsi que gérer la fluidité de l'affichage, a renforcé notre compréhension des interactions entre matériel et logiciel. Bien que les obstacles aient des trous fixes dans cette version, nous aimerions à l'avenir les rendre aléatoires pour rendre le jeu plus dynamique et attrayant.
En conclusion, ce projet a été une opportunité précieuse pour approfondir nos compétences en VHDL, en gestion des ressources FPGA, et en synchronisation des modules graphiques. Malgré les limitations, nous sommes fiers de notre progression et voyons ce travail comme un socle solide pour des projets futurs, incluant des améliorations de gameplay et d'affichage.